



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

AGENT TYPES IN AI

Vacuum Cleaner World

Lab 3 Report

Submitted By:

Name: **Aayush Dhungana**

Roll No: **080BCT004**

Date: **2083-05-18**

Submitted To:

Department of Electronics
and Computer Engineering

Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1	Objectives	3
2	Theoretical Background	3
2.1	Intelligent Agents	3
2.2	Simple Reflex Agent	3
2.3	Model-Based Agent	3
2.4	Vacuum Cleaner World Problem	4
3	Program 1 — Simple Reflex Agent	5
3.1	Problem Statement	5
3.2	Rule Table	5
3.3	Algorithm	5
3.4	Python Implementation	5
3.5	Output	6
3.6	Discussion	6
4	Program 2 — Model-Based Agent	7
4.1	Problem Statement	7
4.2	World Model	7
4.3	Rule Table	7
4.4	Algorithm	7
4.5	Python Implementation	8
4.6	Output	8
4.7	Discussion	9
5	Program 3 — Comparison of Both Agents	10
5.1	Problem Statement	10
5.2	Python Implementation	10
5.3	Output	11
5.4	Discussion	12
6	Conclusion	13

1 Objectives

1. To understand the concept of intelligent agents and their classification in Artificial Intelligence.
2. To implement a **Simple Reflex Agent** for the Vacuum Cleaner World problem.
3. To implement a **Model-Based Agent** with internal memory for the same environment.
4. To compare the behaviour and decision-making of both agent types.
5. To analyse the advantages of maintaining internal state in agent design.

2 Theoretical Background

2.1 Intelligent Agents

An **intelligent agent** is a system that perceives its environment through sensors and acts upon it through actuators. The agent function maps percept sequences to actions. Agents are classified based on their internal architecture and decision-making capabilities.

2.2 Simple Reflex Agent

A **Simple Reflex Agent** is the most basic type of intelligent agent. It selects actions based solely on the **current percept** (sensor input at that moment), using condition-action rules (if-then statements). It has **no memory** of past states.

Architecture: Percept (Current) $\rightarrow$ Condition-Action Rules $\rightarrow$ Action

Key Characteristics:

- Responds only to current environmental stimuli
- Uses predefined rule sets (condition $\rightarrow$ action mapping)
- Does not maintain any internal state or memory
- Works best in fully observable environments
- Extremely fast response time

2.3 Model-Based Agent

A **Model-Based Agent** maintains an **internal state** (memory) to keep track of the world's history and unobserved aspects. It uses a **world model** to make decisions, combining both the current percept and stored knowledge.

Architecture: Percept (Current)+Internal State $\rightarrow$ Update Model $\rightarrow$ Condition-Action Rules $\rightarrow$ Action

Key Characteristics:

- Maintains an internal representation of the environment
- Updates its model based on current percepts
- Can handle partially observable environments
- Remembers past states and actions
- Uses both current percept and internal state for decision making

2.4 Vacuum Cleaner World Problem

The Vacuum Cleaner World is a classic AI problem used to demonstrate agent behaviour.

Table 1: Environment Specifications

Parameter	Description
Rooms	Two rooms: Room A and Room B
States	Each room can be either “Clean” or “Dirty”
Agent Location	The vacuum agent can be in either Room A or Room B
Observability	Fully observable (Simple Reflex) / Partially observable (Model-Based)

Table 2: Possible Actions

Action	Description
Suck	Clean the current room (removes dirt)
Move to A	Navigate from current room to Room A
Move to B	Navigate from current room to Room B
Go to A and Suck	Navigate to Room A and clean it (Model-Based)
Go to B and Suck	Navigate to Room B and clean it (Model-Based)
NoOp / Do Nothing	No operation (idle)

3 Program 1 — Simple Reflex Agent

3.1 Problem Statement

Implement a Simple Reflex Agent for the Vacuum Cleaner World that selects actions based only on the current percept (location and room status), with no memory of past states.

3.2 Rule Table

Table 3: Simple Reflex Agent Rule Table

Condition (Percept)	Action	Rule
Current room is Dirty	Suck	R1
Current room is Clean AND located in Room A	Move to B	R2
Current room is Clean AND located in Room B	Move to A	R3
Any other case (fallback)	NoOp	R4

3.3 Algorithm

1. Start
2. Read current percept (location, status) from sensors
3. IF status equals “Dirty” THEN set action = “Suck”
4. ELSE IF location equals “A” THEN set action = “Move to B”
5. ELSE IF location equals “B” THEN set action = “Move to A”
6. ELSE set action = “NoOp”
7. Return/Output the action
8. End

3.4 Python Implementation

```

1 def simple_reflex_agent(location, status):
2     # location: 'A' or 'B', status: 'Dirty' or 'Clean'
3     if status == "Dirty":
4         return "Suck"
5     elif location == "A":
6         return "Move to B"
7     elif location == "B":
8         return "Move to A"
9     else:
10        return "NoOp"

```

```
11
12 # Percept Sequences to test
13 percepts = [
14     ("A", "Dirty"),
15     ("A", "Clean"),
16     ("B", "Dirty"),
17     ("B", "Clean")
18 ]
19
20 print("***** Simple Reflex Agent *****\n")
21 print("Program by: Aayush Dhungana")
22 print("Roll no: 080BCT004\n")
23
24 for percept in percepts:
25     location, status = percept
26     action = simple_reflex_agent(location, status)
27     print(f"Percept: Location={location}, Status={status} ->
           Action: {action}")
```

Listing 1: Simple Reflex Agent

3.5 Output

```
***** Simple Reflex Agent *****

Program by: Aayush Dhungana
Roll no: 080BCT004

Percept: Location=A, Status=Dirty -> Action: Suck
Percept: Location=A, Status=Clean -> Action: Move to B
Percept: Location=B, Status=Dirty -> Action: Suck
Percept: Location=B, Status=Clean -> Action: Move to A
```

Listing 2: Simple Reflex Agent output

3.6 Discussion

The Simple Reflex Agent correctly responds to all four possible percepts. When a room is dirty, it immediately cleans. When a room is clean, it moves to the other room. The agent has no memory — it does not track which rooms have been previously cleaned. This means it may revisit already-clean rooms unnecessarily.

4 Program 2 — Model-Based Agent

4.1 Problem Statement

Implement a Model-Based Agent for the Vacuum Cleaner World that maintains an internal memory (world model) of room statuses and uses this memory to make decisions.

4.2 World Model

Internal State:

```
world_model = {"A": "Clean", "B": "Clean"}
```

The agent updates `world_model[location]` with each new percept, then queries the model to decide which room needs cleaning.

4.3 Rule Table

Table 4: Model-Based Agent Rule Table

Condition (World Model)	Action
Room A is Dirty	Go to A and Suck
Room B is Dirty	Go to B and Suck
Both Rooms are Clean	Do Nothing, all are clean

4.4 Algorithm

-
1. Start
 2. Initialize `world_model = {"A": "Clean", "B": "Clean"}`
 3. Read current percept (location, status)
 4. Update internal memory: `world_model[location] = status`
 5. IF `world_model["A"] == "Dirty"` THEN action = "Go to A and Suck"
 6. ELSE IF `world_model["B"] == "Dirty"` THEN action = "Go to B and Suck"
 7. ELSE action = "Do Nothing, all are clean"
 8. Return/Output the action
 9. End (internal state persists for next cycle)
-

4.5 Python Implementation

```
1 # Agent's internal memory (model of the world)
2 world_model = {"A": "Clean", "B": "Clean"}
3
4 def model_based_agent(location, status):
5     # Update the agent's memory
6     world_model[location] = status
7
8     # Condition-Action Rules using memory
9     if world_model["A"] == "Dirty":
10         world_model["A"] = "Clean"
11         return "Go to A and Suck"
12     elif world_model["B"] == "Dirty":
13         world_model["B"] = "Clean"
14         return "Go to B and Suck"
15     else:
16         return "Do Nothing, all are clean"
17
18 # Test with percept sequences
19 percepts = [("A", "Dirty"), ("A", "Clean"),
20             ("B", "Dirty"), ("B", "Clean")]
21
22 print("***** Model-Based Agent *****\n")
23 print("Program by: Aayush Dhungana")
24 print("Roll no: 080BCT004\n")
25 print("Initial Memory:", world_model, "\n")
26
27 for location, status in percepts:
28     action = model_based_agent(location, status)
29     print(f"Percept: ({location}, {status})")
30     print(f"    Action: {action}")
31     print(f"    Memory: {world_model}\n")
```

Listing 3: Model-Based Agent

4.6 Output

```
***** Model-Based Agent *****

Program by: Aayush Dhungana
Roll no: 080BCT004

Initially, the state model contains: {'A': 'Clean', 'B': 'Clean'}

The percept sequence is (A, Dirty)
Action: Go to A and Suck
The state model contains: {'A': 'Clean', 'B': 'Clean'}

The percept sequence is (A, Clean)
```



```
Action: Do Nothing, all are clean
The state model contains: {'A': 'Clean', 'B': 'Clean'}

The percept sequence is (B, Dirty)
Action: Go to B and Suck
The state model contains: {'A': 'Clean', 'B': 'Clean'}

The percept sequence is (B, Clean)
Action: Do Nothing, all are clean
The state model contains: {'A': 'Clean', 'B': 'Clean'}
```

Listing 4: Model-Based Agent output

4.7 Discussion

The Model-Based Agent maintains an internal `world_model` dictionary that tracks the status of both rooms. When it perceives a dirty room, it updates the model, cleans the room (sets it to “Clean”), and returns the appropriate action. When both rooms are clean, it correctly idles with “Do Nothing”. The key advantage over the Simple Reflex Agent is that it can reason about rooms it is not currently in.

5 Program 3 — Comparison of Both Agents

5.1 Problem Statement

Compare the behaviour of the Simple Reflex Agent and the Model-Based Agent side by side, highlighting the role of memory in agent decision-making.

5.2 Python Implementation

```

1  # ===== MODEL-BASED AGENT =====
2  world_model = {"A": "Clean", "B": "Clean"}
3
4  def model_based_agent(location, status):
5      world_model[location] = status
6      if world_model["A"] == "Dirty":
7          world_model["A"] = "Clean"
8          return "Go to A and Suck"
9      elif world_model["B"] == "Dirty":
10         world_model["B"] = "Clean"
11         return "Go to B and Suck"
12     else:
13         return "Do Nothing"
14
15 # ===== SIMPLE REFLEX AGENT =====
16 def simple_reflex_agent(location, status):
17     if status == "Dirty":
18         return "Suck"
19     elif location == "A":
20         return "Move to B"
21     elif location == "B":
22         return "Move to A"
23     else:
24         return "NoOp"
25
26 # ===== COMPARISON TABLE =====
27 percepts = [("A", "Dirty"), ("A", "Clean"),
28             ("B", "Dirty"), ("B", "Clean")]
29
30 print("=" * 70)
31 print("COMPARISON")
32 print("=" * 70)
33 print(f"\n{'Percept':<15} {'Model-Based':<25} {'Simple Reflex':<20}")
34 print("-" * 70)
35
36 world_model = {"A": "Clean", "B": "Clean"}
37 for location, status in percepts:
38     world_model = {"A": "Clean", "B": "Clean"}
39     mb_action = model_based_agent(location, status)
40     sr_action = simple_reflex_agent(location, status)

```

```
41 print(f"({location}, {status:<5}) {mb_action:<25} {
    sr_action}")
```

Listing 5: Comparison of both agents

5.3 Output

```
=====

MODEL-BASED AGENT (With Memory)
=====

Percept          Action                                     Memory After
-----

(A, Dirty)  Go to A and Suck                {'A': 'Clean', 'B':
'Clean'}
(A, Clean)  Do Nothing                        {'A': 'Clean', 'B':
'Clean'}
(B, Dirty)  Go to B and Suck                {'A': 'Clean', 'B':
'Clean'}
(B, Clean)  Do Nothing                        {'A': 'Clean', 'B':
'Clean'}

=====

SIMPLE REFLEX AGENT (No Memory)
=====

Percept          Action
-----

(A, Dirty)  Suck
(A, Clean)  Move to B
(B, Dirty)  Suck
(B, Clean)  Move to A

=====

COMPARISON
=====

Percept          Model-Based                                     Simple Reflex
-----

(A, Dirty)  Go to A and Suck                Suck
(A, Clean)  Do Nothing                        Move to B
```

```

(B, Dirty)  Go to B and Suck          Suck
(B, Clean)  Do Nothing                Move to A

=====

KEY DIFFERENCE
=====

Model-Based Agent: Has MEMORY - remembers room statuses
Simple Reflex Agent: NO MEMORY - acts only on current percept

Example: After cleaning Room A, when seeing (A, Clean):
- Model-Based: Does Nothing (remembers A is clean)
- Simple Reflex: Moves to B (doesn't remember)

```

Listing 6: Comparison output

5.4 Discussion

The comparison reveals the fundamental difference between the two agent types:

Table 5: Agent Comparison Summary

Feature	Simple Reflex Agent	Model-Based Agent
Memory	No memory	Maintains internal world model
Decision Basis	Current percept only	Current percept + stored state
Room A Clean	Moves to B (always)	Does Nothing (remembers it's clean)
Room B Clean	Moves to A (always)	Does Nothing (remembers it's clean)
Efficiency	May revisit clean rooms	Avoids unnecessary actions
Complexity	Simple	More complex

The Model-Based Agent is more efficient because it remembers which rooms are clean and avoids redundant cleaning operations. The Simple Reflex Agent, while simpler, may waste actions moving to already-clean rooms.

6 Conclusion

This lab demonstrated the implementation and comparison of two fundamental agent types in Artificial Intelligence:

1. **Simple Reflex Agent:** Acts solely on the current percept using condition-action rules. It is simple and fast but lacks memory, leading to potentially redundant actions in environments where state tracking is beneficial.
2. **Model-Based Agent:** Maintains an internal world model that persists across percept cycles. This memory allows the agent to make more informed decisions by considering both current observations and past knowledge.
3. **Comparison:** The Model-Based Agent outperforms the Simple Reflex Agent in scenarios where maintaining state information leads to more efficient behaviour. The key trade-off is between simplicity (Reflex) and intelligence (Model-Based).

The Vacuum Cleaner World problem effectively illustrates how adding internal state to an agent architecture significantly improves its decision-making capabilities, a fundamental principle in AI agent design.